

Analisi e Progettazione del Software

Esempi di progettazione a oggetti con i pattern GRASP

Oliviero Riganelli

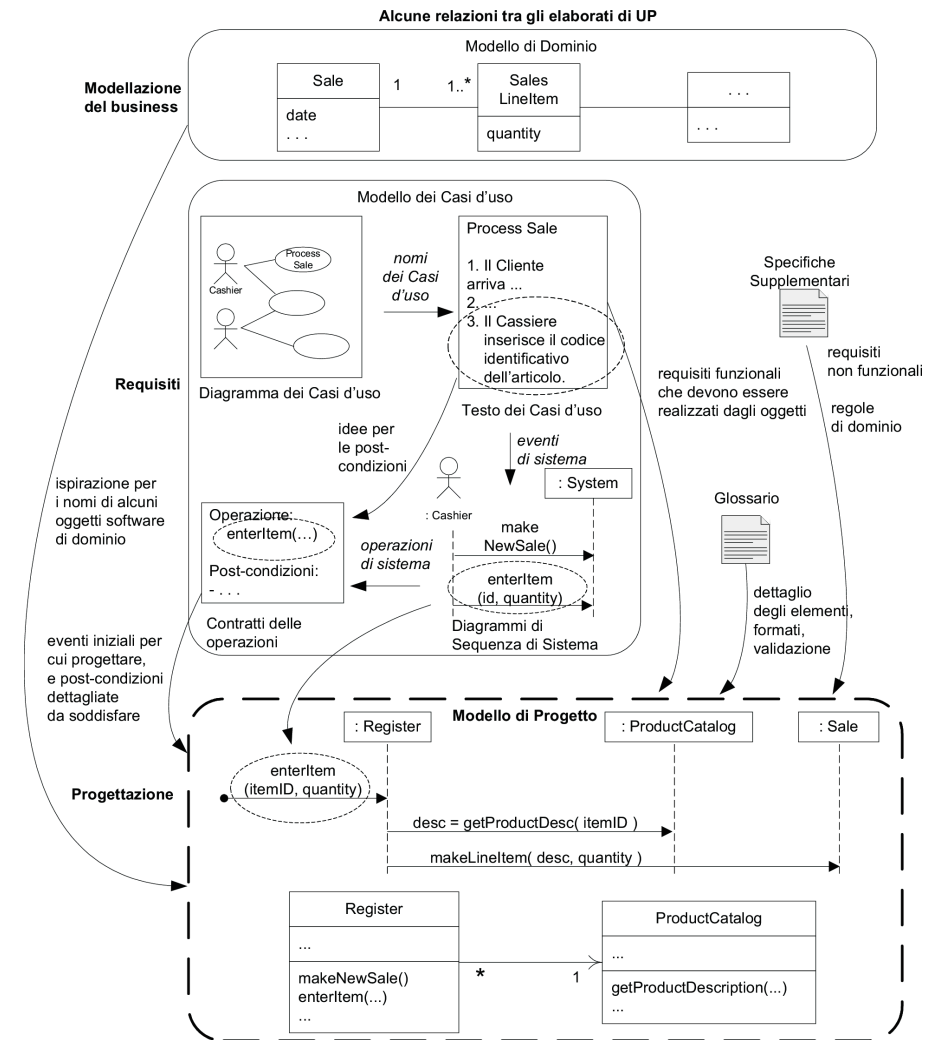
Realizzazione dei casi d'uso

Una **realizzazione di caso d'uso** descrive come viene realizzato un caso d'uso all'interno del Modello di Progetto, in termini di oggetti che collaborano

- progetto relativo a uno o più scenari di un caso d'uso
 - *illustrato da diagrammi di interazione*
- “realizzazione di caso d'uso” è un termine di UP
 - *enfatica la connessione forte tra requisiti e progetto: il progetto deve soddisfare i requisiti*

Influenza tra elaborati

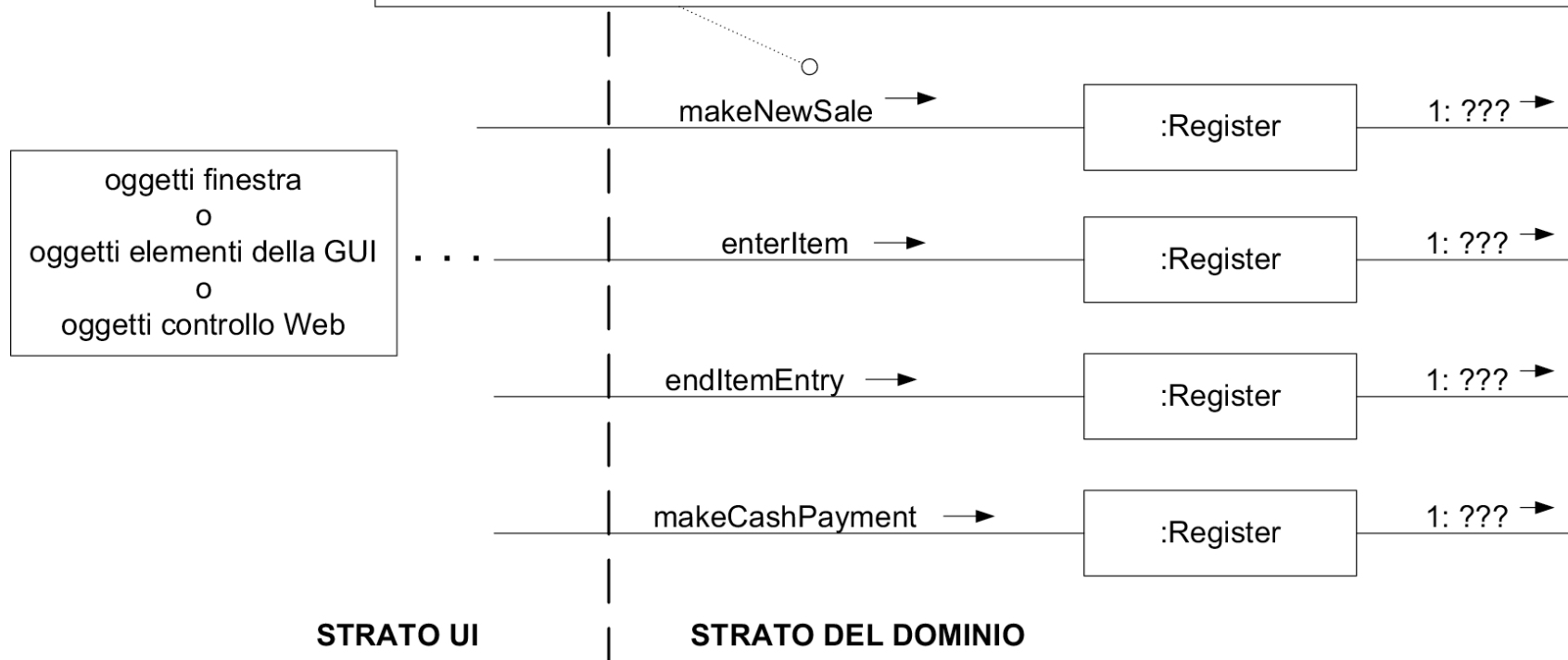
- Il caso d'uso suggerisce le operazioni di sistema, mostrate negli SSD
- Le operazioni di sistema diventano i messaggi iniziali entranti nei controller per i diagrammi di interazione
- I diagrammi di interazione per lo strato del dominio illustrato come gli oggetti interagiscono per soddisfare i compiti richiesti (realizzazione dei casi d'uso)



Diagrammi di interazione e gestione delle operazioni di sistema

makeNewSale, e così via, sono le operazioni di sistema dall'SSD

ciascun diagramma di interazione principale inizia con un'operazione di sistema che arriva a un oggetto controller dello strato del dominio, come *Register*, e poi prosegue mostrando l'interazione tra gli oggetti di dominio per gestire quell'operazione di sistema



Casi d'uso e realizzazione dei casi d'uso

- Una realizzazione di caso d'uso può essere progettata direttamente sul testo del caso d'uso
 - *insieme agli altri elaborati dove sono espressi i relativi requisiti (es. Specifiche Supplementari)*
- i requisiti scritti sono imperfetti, e spesso molto imperfetti
 - *Coinvolgere frequentemente il cliente*

Modello di dominio e realizzazione dei casi d'uso

- Nei diagrammi di interazione sono mostrati oggetti software
- Per sostenere un salto rappresentazionale basso, il progettista sceglie alcuni di questi oggetti software ispirandosi al modello il dominio
- Il modello di dominio è una fonte d'ispirazione per la progettazione, ma non un vincolo!
- Durante il lavoro di progettazione è normale
 - *scoprire nuove classi concettuali che erano sfuggite durante l'analisi*
 - *creare classi software i cui nomi e il cui scopo non sono per niente correlate al modello dominio*

Contratti delle operazioni e realizzazioni di caso d'uso

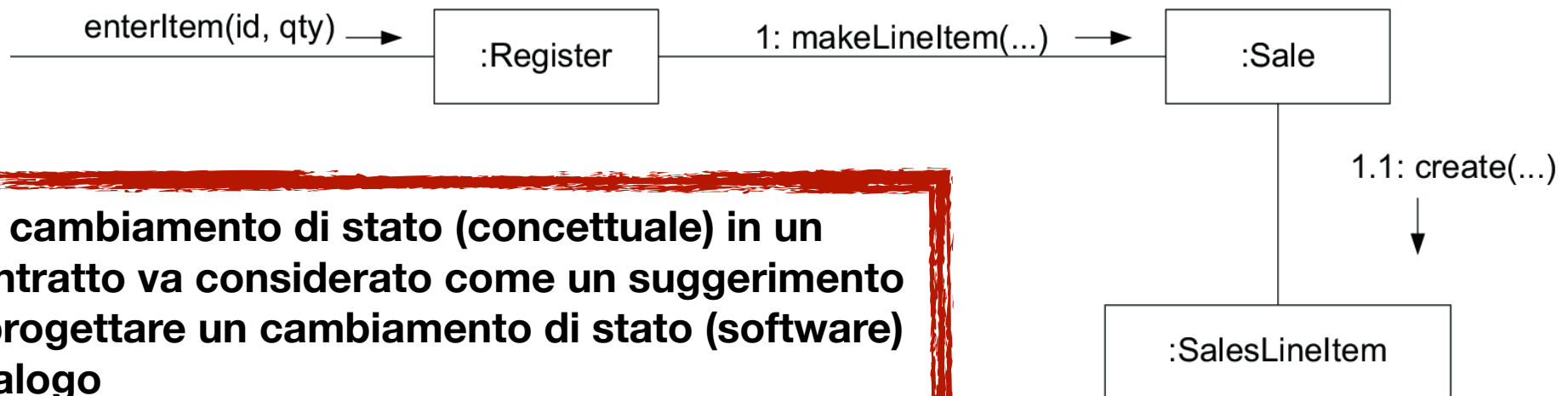
- Per alcune operazioni di sistema complesse, è possibile che siano stati scritti contratti che possono aiutare la realizzazione dei casi d'uso
 - *Per ogni contratto si considera i cambiamenti di stato descritti dalle post condizioni*
 - *si progettano interazioni di messaggi per soddisfare i requisiti*

Contratto CO2: enterItem

Operazione:	enterItem(itemID : ItemID, quantity : integer)
Riferimenti:	Casi d'Uso: Elabora Vendita
Pre-condizioni:	è in corso una vendita s.
Post-condizioni:	- è stata creata un'istanza sli di SalesLineItem. - ...

Contratti delle operazioni e realizzazioni di caso d'uso

- Si crea il diagramma di un'interazione parziale che soddisfa il cambiamento di stato relativo alla creazione dell'istanza di SalesLineItem



Un cambiamento di stato (concettuale) in un contratto va considerato come un suggerimento a progettare un cambiamento di stato (software) analogo

Progettazione per l'operazione MakeNewSale

- L'operazione di sistema makeNewSale avviene quando un cassiere inizia una richiesta per avviare una nuova vendita

Contratto CO1: makeNewSale

Operazione:	makeNewSale()
--------------------	---------------

Riferimenti:	Casi d'Uso: Elabora Vendita
---------------------	-----------------------------

Pre-condizioni:	nessuna.
------------------------	----------

Post-condizioni:	– è stata creata un'istanza s di Sale. – s è stata associata con il Register. – gli attributi di s sono stati inizializzati.
-------------------------	--

Scelta della classe controller

- La prima scelta di progetto riguarda la scelta del controller per il messaggio dell'operazione sistema `makeNewSale`

Un oggetto che rappresenta il “sistema” complessivo, “l’oggetto radice”, un dispositivo speciale, un punto d’accesso o un sottosistema principale.

Store. Una specie di oggetto radice, poiché la maggior parte degli altri oggetti di dominio è considerata “all’interno” di *Store*.

Register. Un dispositivo specializzato su cui è eseguito il software; chiamato anche *POS-Terminal*.

POSSystem. Un nome che suggerisce il sistema complessivo.

Un oggetto che rappresenta un ricevitore o un gestore di tutti gli eventi di sistema di uno scenario di caso d’uso.

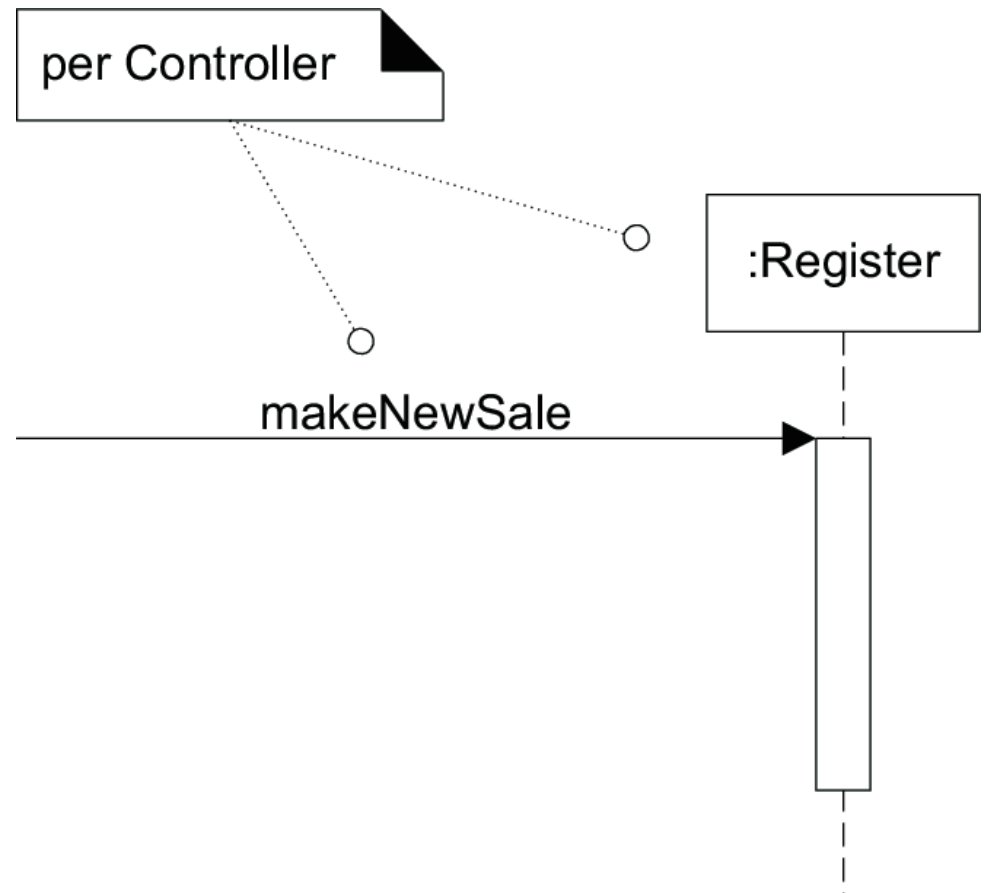
ProcessSaleHandler. Costruito secondo il pattern <use-case-name> “Handler” o “Session”.

ProcessSaleSession.

Scelta della classe controller

Register

- ispirato al modello di dominio
- ora indica un oggetto software nel modello di progetto
- rappresenta (ma non è) il registratore di cassa – ovvero, un punto d'accesso al sistema da parte dell'attore primario del caso d'uso in discussione



Prossima Responsabilità

- Dopo aver assegnato una responsabilità, la domanda successiva è come fa l'oggetto a cui ho assegnato una responsabilità a soddisfare la responsabilità che gli è stata assegnata?
 - *che cosa deve fare il Register per soddisfare la responsabilità dell'operazione di sistema makeNewSale?*

Contratto CO1: makeNewSale

Operazione: makeNewSale()

Riferimenti: Casi d'Uso: Elabora Vendita

Pre-condizioni: nessuna.

Post-condizioni:

- è stata creata un'istanza s di Sale.
- s è stata associata con il Register.
- gli attributi di s sono stati inizializzati.

Creazione di una vendita

Il contratto dell'operazione di sistema makeNewSale ha, tra le sue post-condizioni, la creazione di un'istanza s di Sale

- il contratto parla della creazione di un oggetto Sale concettuale
- per salto rappresentazione basso, questo cambiamento può essere gestito con la creazione di un oggetto Sale software

Creazione di una vendita

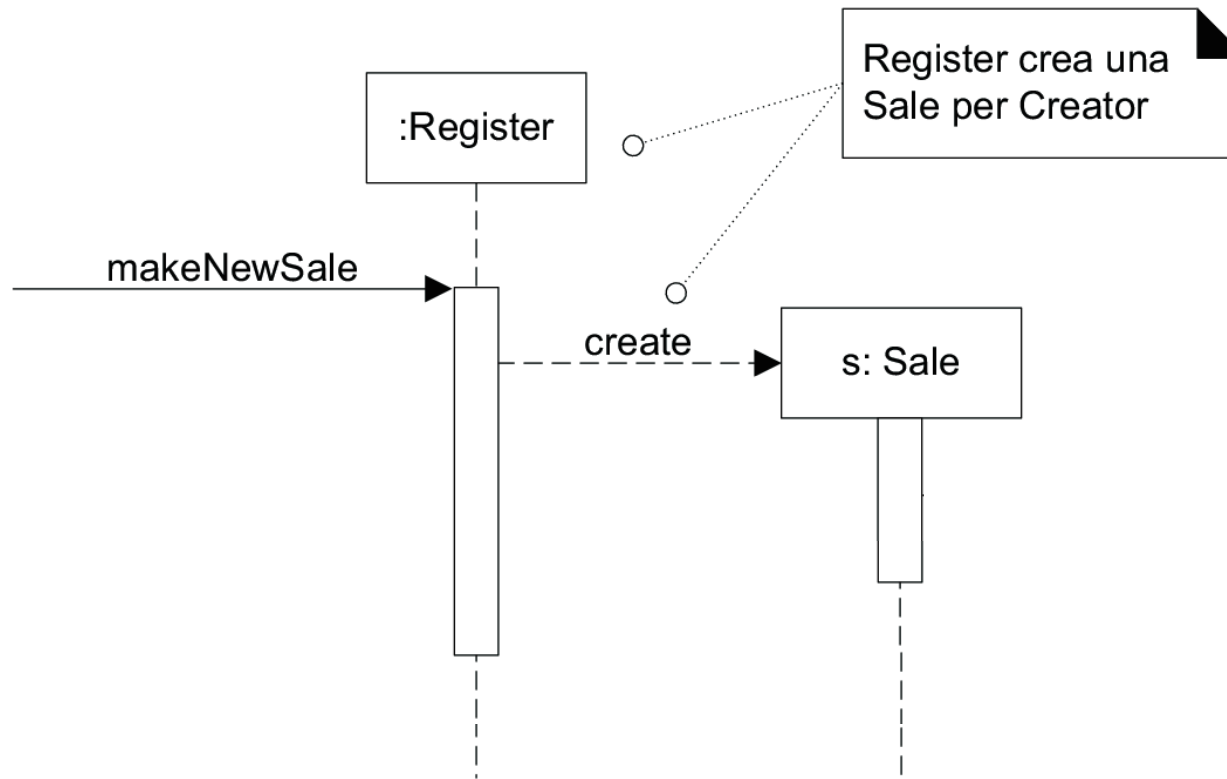
- Chi deve creare un oggetto software Sale?

Nome:	Creator (Creatore)
Problema:	Chi crea un oggetto A?
Soluzione (può essere vista come un consiglio):	Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è): • B “contiene” o aggrega con una composizione oggetti di tipo A. • B registra A. • B utilizza strettamente A. • B possiede i dati per l'inizializzazione di A.

Applicando il pattern Creator

- Analizzando il modello di dominio si scopre che è un Register può essere considerato come registratore di una Sale
- Secondo il modello di dominio, anche lo Store registra vendite
 - *Tuttavia, lo Store registra solo vendite completate, ma la sale da creare va considerata, al momento, solo un tentativo di vendita*
- Register dovrà utilizzare strettamente la Sale corrente

Register crea una Sale



Prossima Responsabilità

- come collegare la Sale al Register?

Contratto CO1: makeNewSale

Operazione:	makeNewSale()
--------------------	---------------

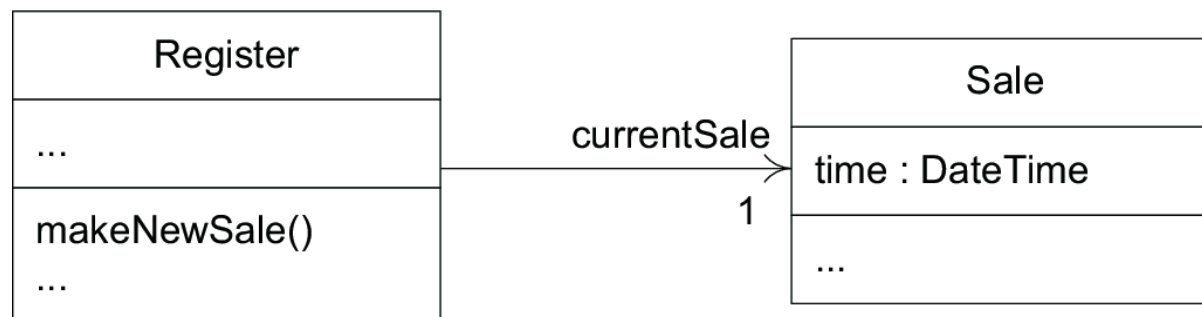
Riferimenti:	Casi d'Uso: Elabora Vendita
---------------------	-----------------------------

Pre-condizioni:	nessuna.
------------------------	----------

Post-condizioni:	– è stata creata un'istanza s di Sale. – s è stata associata con il Register. – gli attributi di s sono stati inizializzati.
-------------------------	--

Collegare la Sale al Register

- Register mantiene un riferimento all'istanza corrente di Sale nell'attributo currentSale



Prossima responsabilità

- La Sale deve creare una collezione vuota per memorizzare tutte le future istanze di SaleLineItems che verranno aggiunta alla vendita
- In base a Creator, Sale è un buon candidato per la collezione vuota si SalesLineItems (Sale contiene SaleLineItems)

Contratto CO1: makeNewSale

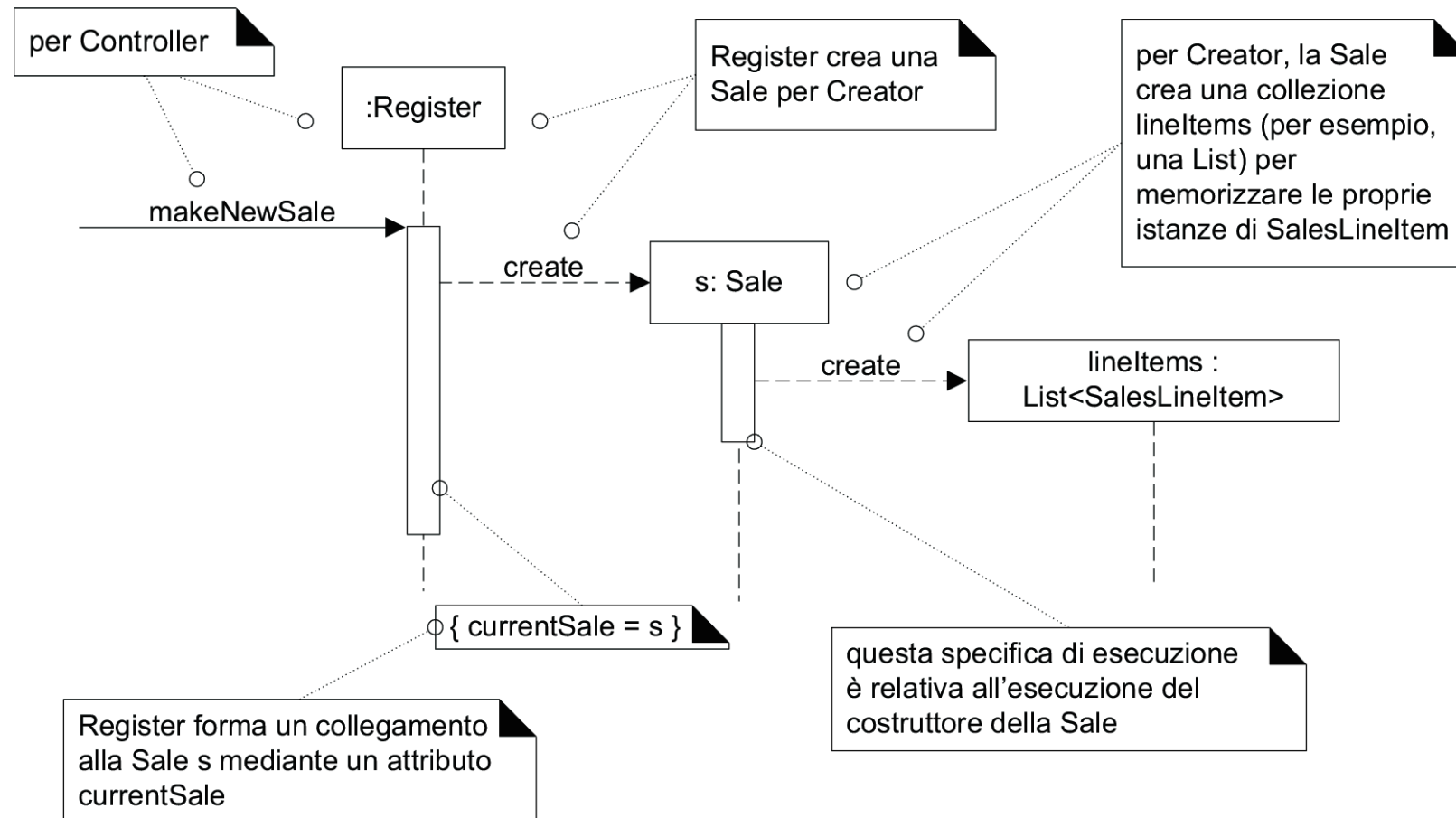
Operazione:	makeNewSale()
--------------------	---------------

Riferimenti:	Casi d'Uso: Elabora Vendita
---------------------	-----------------------------

Pre-condizioni:	nessuna.
------------------------	----------

Post-condizioni:	– è stata creata un'istanza s di Sale. – s è stata associata con il Register. – gli attributi di s sono stati inizializzati.
-------------------------	--

Diagramma di interazione per makeNewSale



DCD parziale relativo alla progettazione statica per makeNewSale



Progettazione a oggetti

- chiediti qual è una responsabilità da assegnare
- sulla base del tipo di responsabilità, seleziona il pattern GRASP da applicare
- sulla base del pattern, scegli l'oggetto a cui assegnare la responsabilità
- chiediti come fa l'oggetto scelto a soddisfare la responsabilità
 - *questo può portare all'identificazione di altre responsabilità da assegnare*

Progettazione per l'operazione enterItem

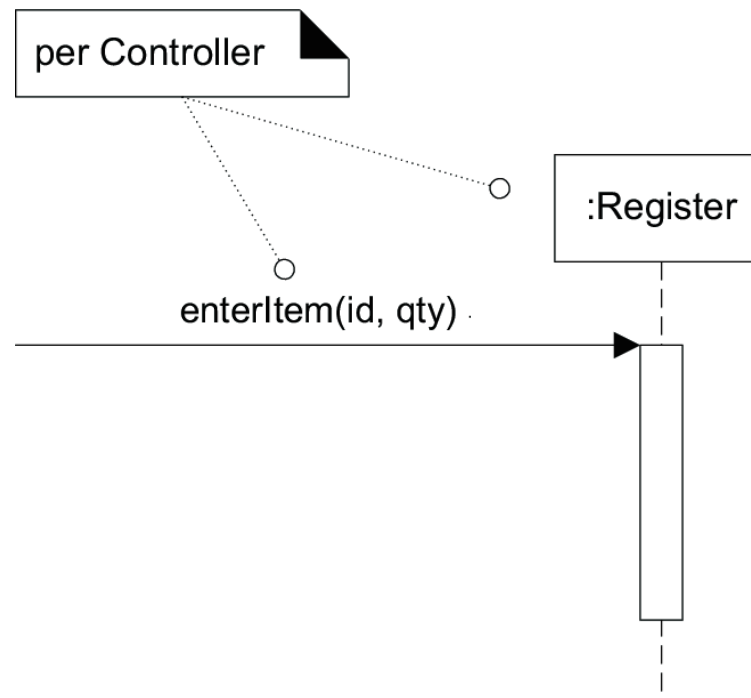
L'operazione di sistema enterItem avviene quando un cassiere inserisce l'itemID e (opzionalmente) la quantità di un articolo da acquistare

Contratto CO2: enterItem

Operazione:	enterItem(itemID: ItemID, quantity: Integer)
Riferimenti:	Casi d'uso: Elabora Vendita
Pre-condizioni:	è in corso una vendita s.
Post-condizioni:	– è stata creata un'istanza sli di SalesLineItem. – sli è stata associata con la Sale corrente s. – sli è stata associata con una ProductDescription, in base alla corrispondenza con itemID. – sli.quantity è diventata quantity.

Scelta della classe controller

- Il pattern controller suggerisce di utilizzare la stessa classe controller per tutte l'operazioni di sistema di un caso d'uso



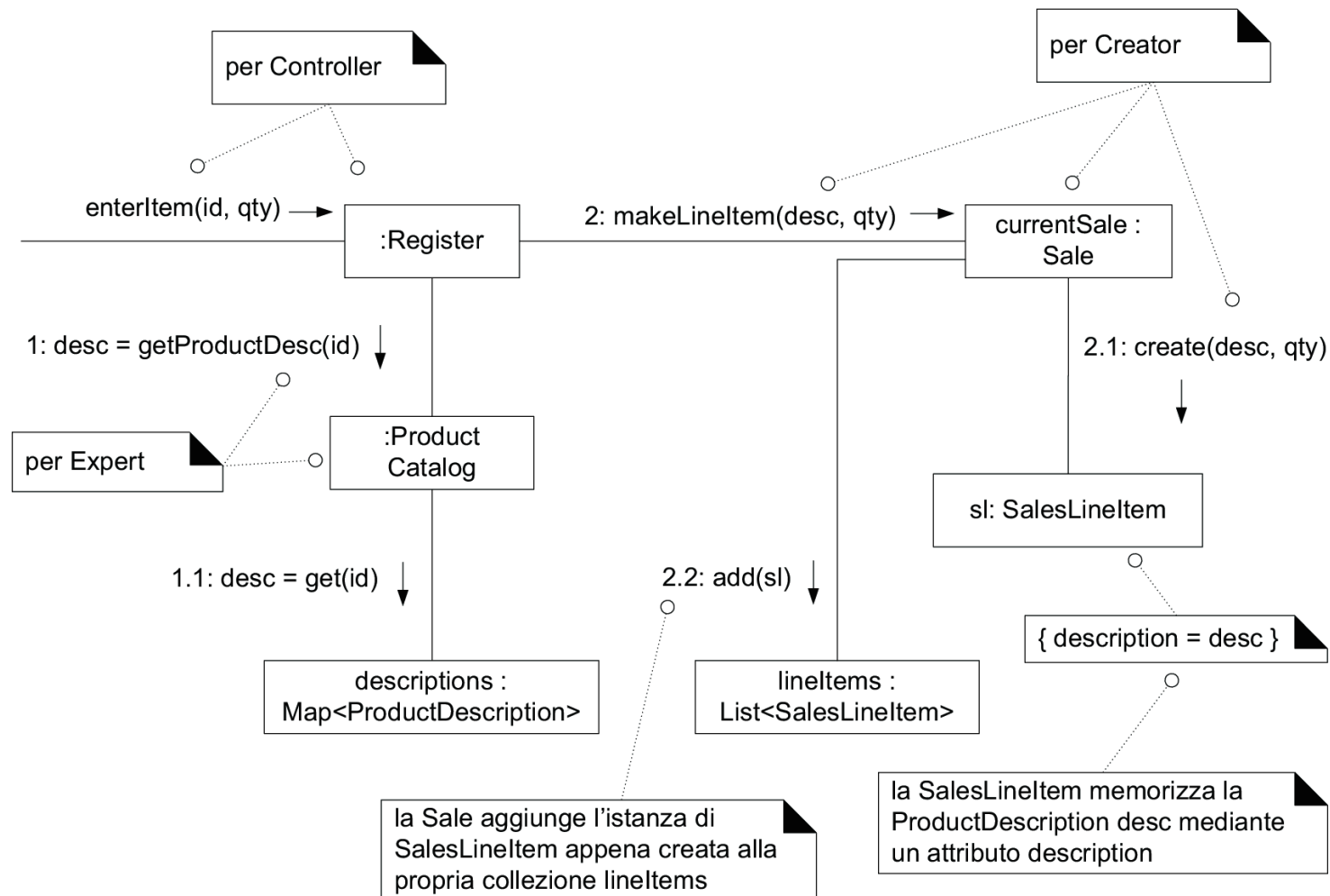
Creazione di una nuova SalesLineItem

- Chi crea il nuovo oggetto SalesLineItem?
 - *Per Creator, Sale software è un candidato appropriato per la creazione di SalesLineItem in quanto contiene oggetti SaleLineItems*
 - *La Sale può essere poi collegata con la SaleLineItems appena creata memorizzandola nella sua collezione di righe di articoli*
 - *La nuova SaleLineItems quando viene creata
 - *va collegata a una ProductDescription*
 - *Richiede una quantità**
 - *Per Creator, viene inviato un messaggio makeLineItem a una Sale affinché essa crei una SaleLineItem*
 - *I parametri per il messaggio makeLineItem comprendono ProductDescription che corrisponde all'itemID e la quantity in modo che SaleLineItem possa memorizzarli*

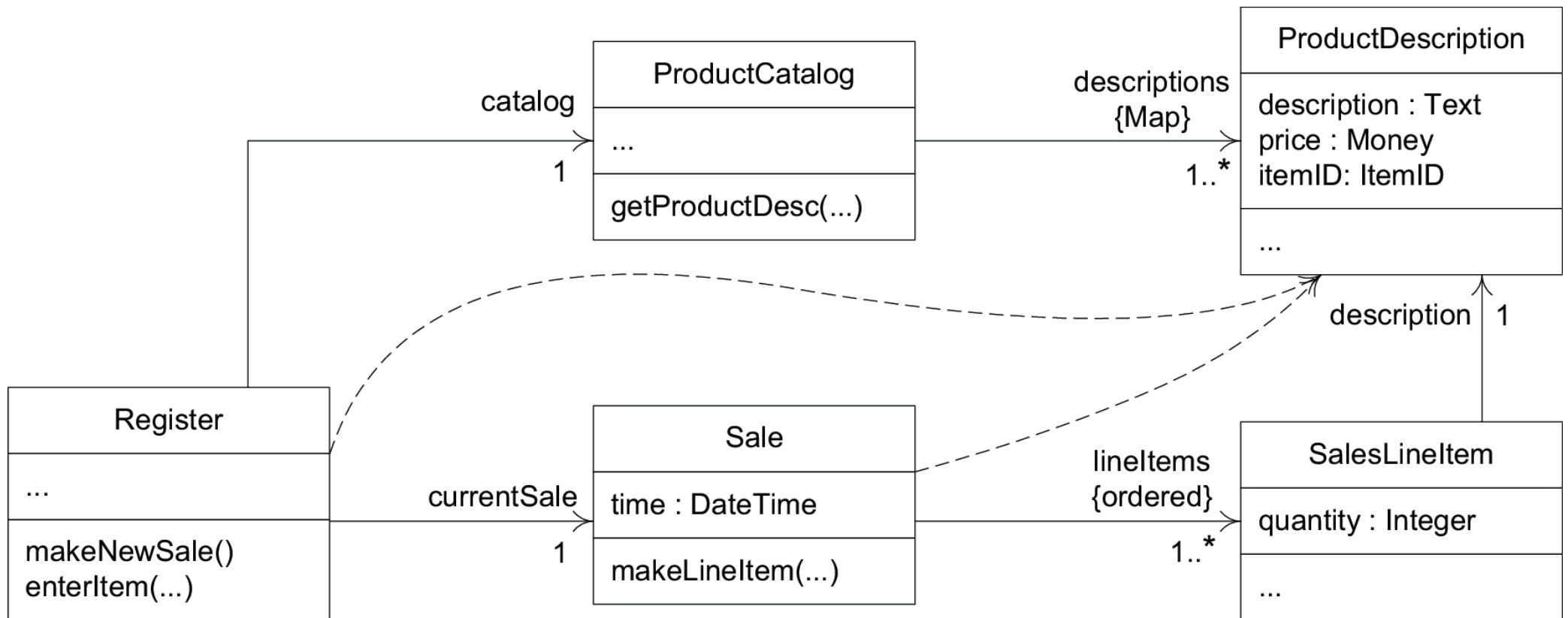
Ricerca di una ProductDescription

- Chi deve essere responsabile della conoscenza di una ProductDescription, in base a una corrispondenza con itemID?
- Analizzando il modello di dominio, ProductCatalog contiene logicamente tutte le ProductDescription
 - *Un ProductCatalog software conterrà oggetti ProductDescription software*
- Per Expert, ProductCatalog è un buon candidato per questa responsabilità

Diagramma di interazione per enterItem



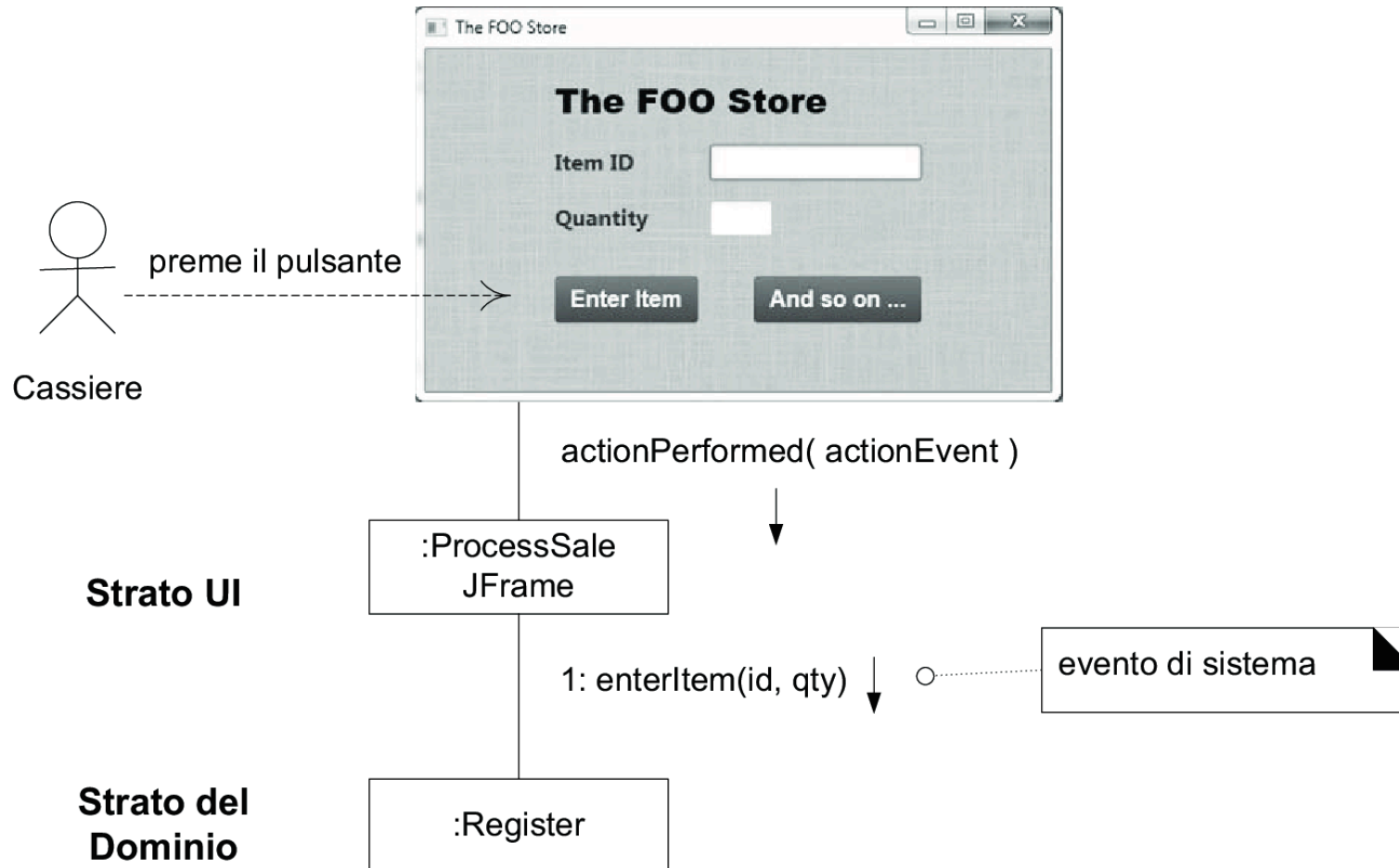
DCD parziale relativo alla progettazione svolta finora



Come collegare lo strato UI allo strato del dominio

- Scelte progettuali comuni:
 - *Un oggetto inizializzazione (per esempio un oggetto Factory) chiamato dal metodo iniziale dell'applicazione che crea sia un oggetto UI che un oggetto di dominio e passa l'oggetto di dominio all'oggetto UI*
 - *Un oggetto UI che recupera l'oggetto di dominio da una sorgente nota come un oggetto factory responsabile della creazione di oggetti di dominio*

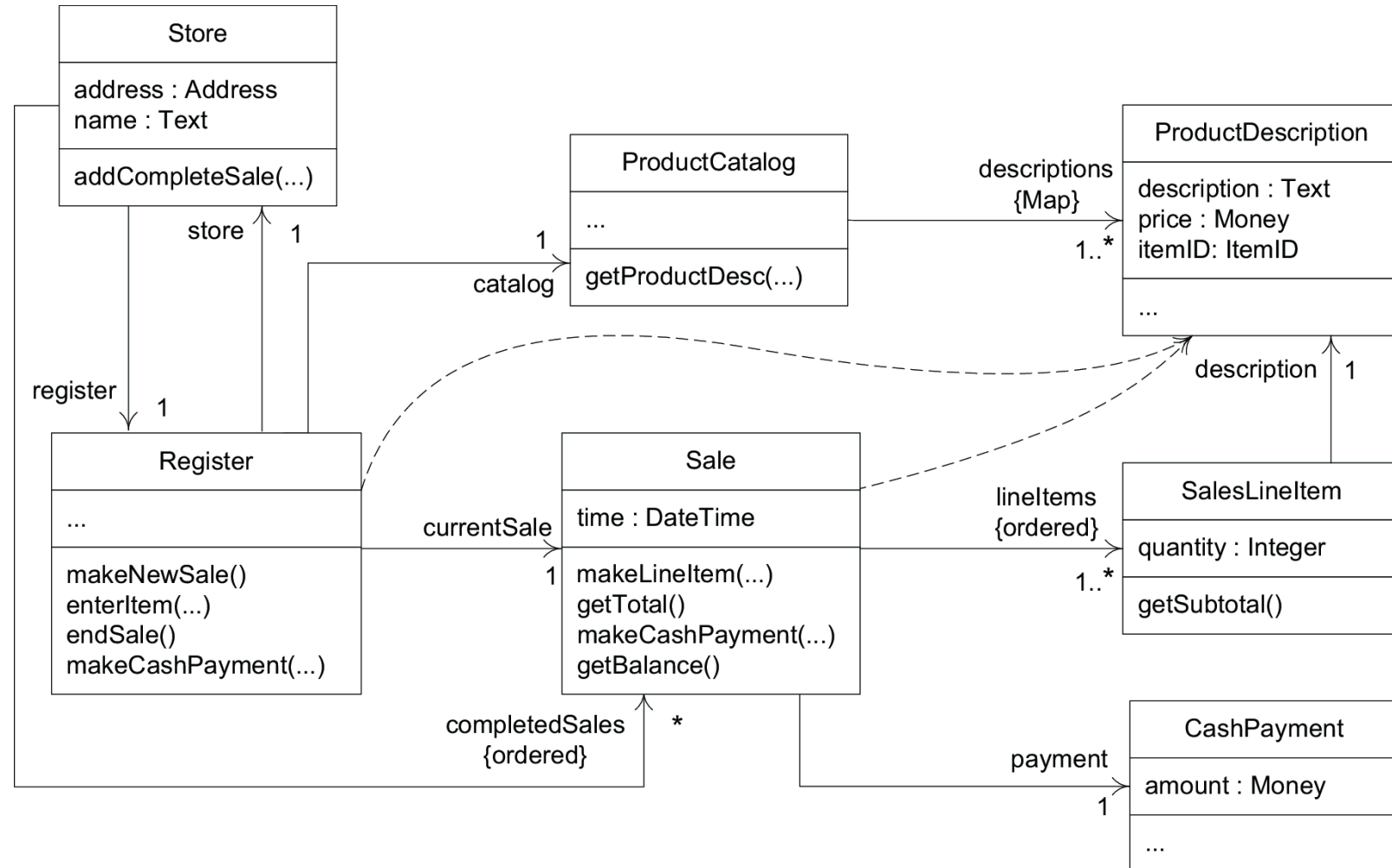
Come collegare lo strato UI allo strato del dominio



Un DCD più completo

Vedere il libro per la
progettazione
dell'operazioni

- endItemEntry
- makeCashPayment



Inizializzazione e caso d'uso d'avviamento

- Molti sistemi, se non tutti, hanno un caso d'uso d'avviamento
 - svolge le operazioni iniziali per l'avvio del sistema
 - comprende in particolare la creazione e l'inizializzazione di molti “oggetti radice” o degli oggetti di lunga vita
- l'operazione di sistema startUp è la prima ad essere eseguita
 - di solito però non è la prima ad essere progettata

Come sono avviate le applicazioni

Come progettare per il caso d'uso di avviamento?

- un idioma progettuale comune è quello di creare un oggetto di dominio iniziale
 - *il primo oggetto software di dominio ad essere creato*
 - *responsabile della creazione degli altri oggetti di dominio che sono inizialmente necessari*
- quale oggetto di dominio iniziale per il sistema POS NextGen?
 - *Store*

Come sono avviate le applicazioni

```
public class Main {  
  
    public static void main(String[] args) {  
  
        /* Store è l'oggetto di dominio iniziale.  
        * Lo Store crea degli altri oggetti di dominio. */  
  
        Store store = new Store();  
  
        /* Avvia l'interfaccia grafica, passandogli il controller. */  
  
        Register register = store.getRegister();  
  
        ProcessSaleJFrame frame = new ProcessSaleJFrame(register);  
  
        ...  
    }  
  
    ...  
}
```

Scelta dell'oggetto di dominio iniziale

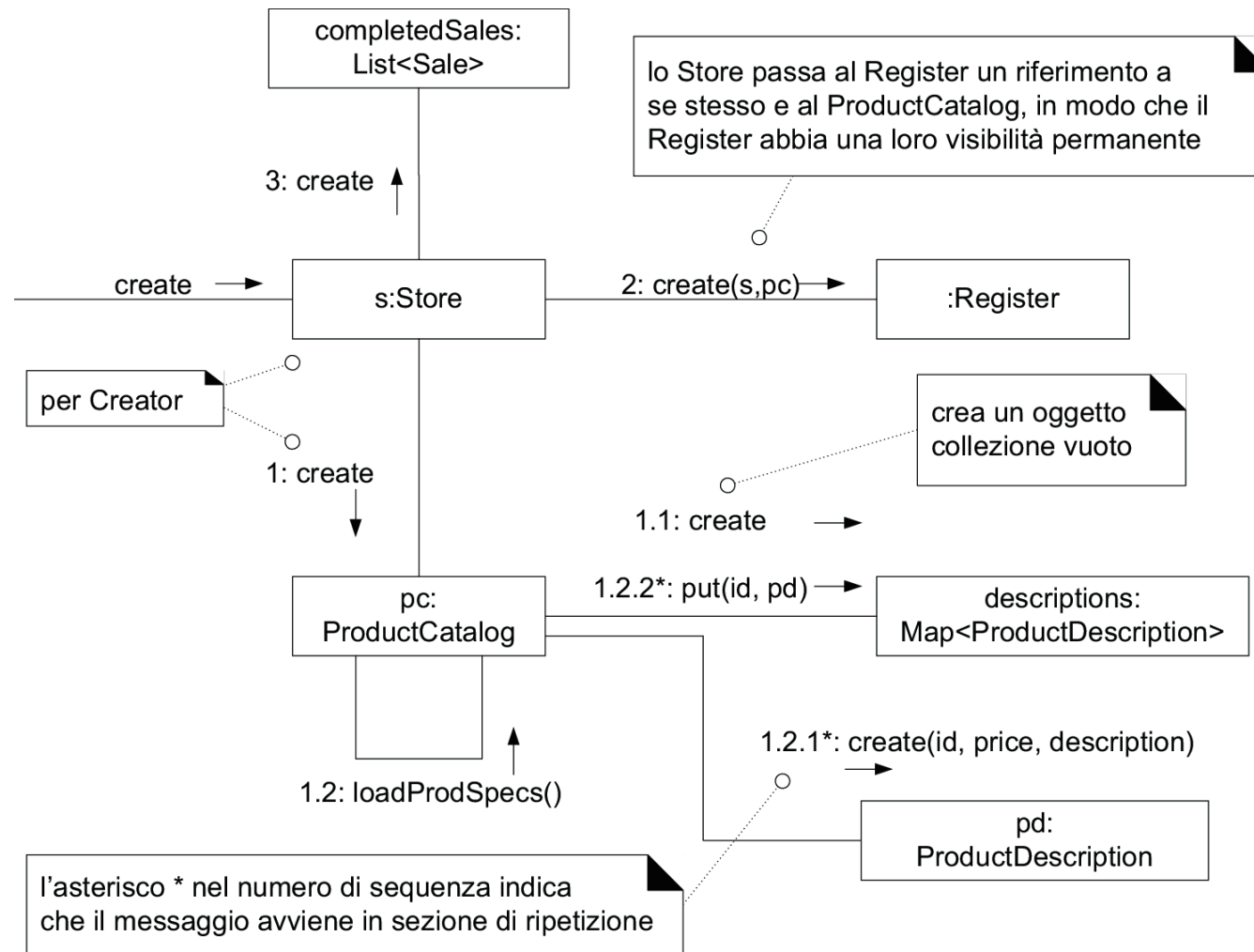
- L'oggetto di dominio iniziale va scelto tra quelli che sono vicini alla radice della gerarchia di contenimento/aggregazione degli oggetti di dominio
 - *un oggetto che contiene (direttamente o indirettamente) tutti o molti altri oggetti*
 - *oppure, un facade controller*
 - *scegliendo anche sulla base di High Cohesion e Low Coupling*
- Quale oggetto di dominio iniziale per il sistema POS NextGen?
 - *Store*

Progettazione per Store.create

I compiti di creazione e inizializzazione derivano dalle necessità del lavoro di progettazione precedente:

- Vanno creati uno Store, un Register, un ProductCatalog e gli oggetti ProductDescription, nonché la collezione di Sale completate
- Le ProductDescription vanno associate al ProductCatalog (per enterItem)
- Il ProductCatalog va associato al register (per enterItem)
- Il Register va associato allo Store (per main)
- Lo Store va associato al Register (per makeCashPayment)

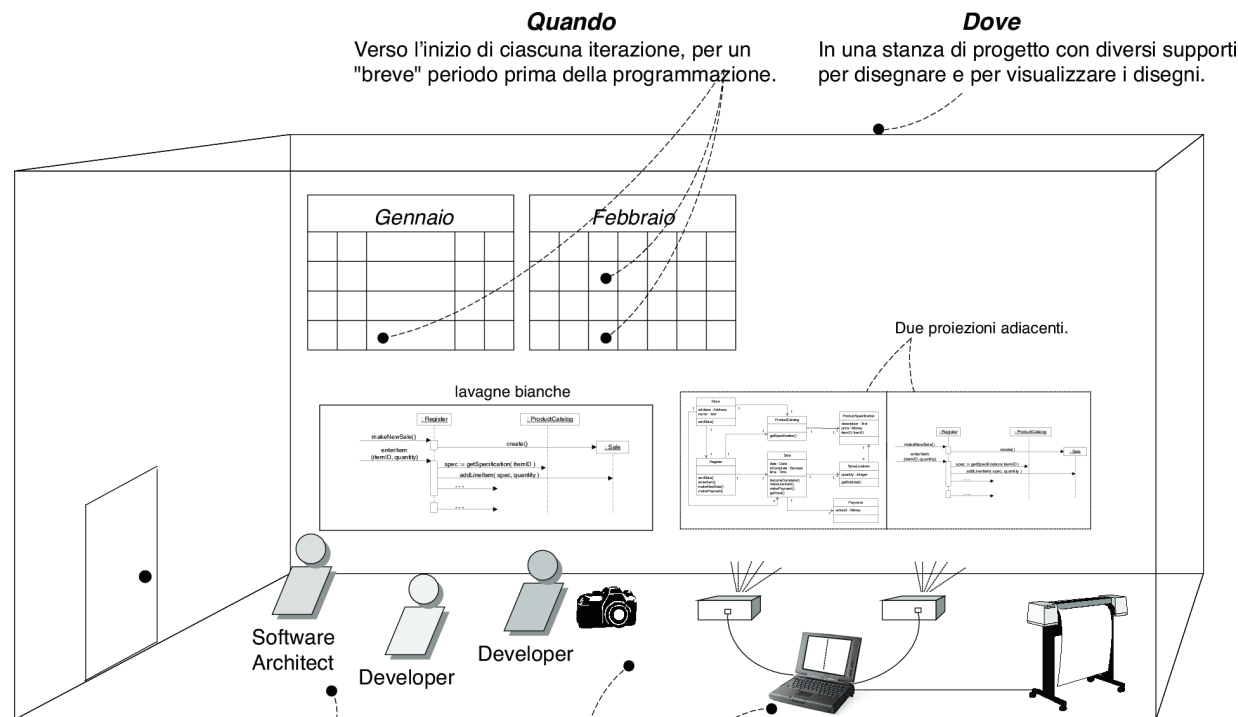
Creazione dell'oggetto di dominio iniziale e degli oggetti successivi



Realizzazioni di caso d'uso per l'iterazione 1 per Monopoly

- si veda il libro

Progettazione a oggetti iterativa ed evolutiva



Progettazione a oggetti all'interno di UP

Disciplina	Elaborato Iterazione →	Ideaz. I1	Elab. E1..En	Costr. C1..Cn	Trans. T1..T2
Modellazione del business	Modello di Dominio		i		
Requisiti	Modello dei Casi d'Uso (SSD)	i	r		
	Specifiche Supplementari	i	r		
	Glossario	i	r		
Progettazione	Modello di progetto		i	r	
	Documento dell'Architettura Software		i		
	Modello dei Dati		i	r	

Riepilogo

- La progettazione di oggetti è basata su
 - *progettazione di interazioni tra oggetti*
 - *identificazione e assegnazione di responsabilità*
- Le scelte progettuali hanno un impatto profondo sulle qualità del sistema
 - *estendibilità, chiarezza, mantenibilità, riusabilità*
- Ci sono principi su cui basare le scelte progettuali
- i pattern GRASP riassumono alcune delle scelte per l'assegnazione di responsabilità più generali e più comuni usate dai progettisti orientati agli oggetti